

Esame Software Engineering (AA 2025/26)

19 Febbraio 2026 - Lab. Colossus - Via salaria 113

Enrico Tronci

*Computer Science Department, Sapienza University of Rome
Via Salaria 113 - 00198 Roma - Italy*

`tronci@di.uniroma1.it`

`https://raise.uniroma1.it`

Esercizio 3 (20 punti)

La comunicazione tra i componenti del sistema descritto avviene attraverso la rete che può essere modellata come negli esempi visti a lezione. La rete impiega un tempo r per leggere un messaggio da una coda di input ed un tempo w per scrivere un messaggio in una coda di output.

Un sistema di *e-commerce* ha C customers, S servers, F fornitori ed un DB centralizzato.

Ciascun *customers* può essere modellato come una catena di Markov con un solo stato (0). Ciascun customer resta nello stato 0 per un tempo τ scelto uniformemente a random nell'intervallo $[A, B]$.

Se il customer decide di fare una transizione (che lo riporta nello stato 0) allora:

1. sceglie un server s a random tra S server (numerati $1, 2, \dots S$);
2. sceglie un prodotto i a random tra P prodotti numerati $1, 2, \dots P$.
3. sceglie a random la quantità q da richiedere per il prodotto i . Il valore q è un intero positivo minore od uguale a Q .

Questo vuol dire che il customer ha richiesto al server s , q items del prodotto i .

Si noti che quando il customer permane nello stato 0 nulla viene inviato ai servers.

I servers sono tutti uguali. Quando un server riceve una richiesta (i, q) dal customer j esegue le seguenti operazioni:

1. invia (query) al DB centralizzato il valore i ;
2. il DB risponde al server con il valore g di items di tipo i disponibili;
3. il server invia al DB il valore k di items che invierà al customer j , quindi il DB deve decrementare g di k ;
4. il server invia al customer j la coppia (i, k) , dove $k = q$ se ci sono almeno q item del prodotto i nel DB centralizzato ($g \geq q$), altrimenti k è uguale al numero di items del prodotto i disponibili nel DB (cioè g).

Un server impiega tempo z per leggere l'input e tempo v per mandare un output.

Ciascun *fornitore* può essere modellato come una catena di Markov con un solo stato (0). Ciascun fornitore resta nello stato 0 per un tempo γ scelto uniformemente a random nell'intervallo $[V, W]$.

Se il fornitore decide di fare una transizione (che lo riporta nello stato 0) allora invia al DB centralizzato il messaggio (i, q) dove

1. il prodotto i è scelto a random tra P prodotti numerati $1, 2, \dots, P$;
2. la quantità q del prodotto i è scelta a random in $\{1, 2, \dots, Q\}$;

Il DB centralizzato riceve prodotti dai fornitori e richieste dai servers. Il DB centralizzato impiega un tempo l per leggere un messaggio dalla sua coda di input dalla rete ed impiega un tempo s per inviare un messaggio sulla sua coda di output verso la rete.

Per ognugno dei P prodotti, il DB è inizializzato a random con valori compresi tra 0 e Q .

Una *transazione* è una richiesta da un customer ad un server.

Si consideri una transazione in cui un customer invia al server la richiesta (i, q) e riceve dal server la risposta (i, k) . Le mancate vendite m associate alla transazione sono $m = (q - k)$. Poichè $k \leq q$ abbiamo che m è il numero di items che il server non ha potuto inviare al customer.

Il numero di *mancate vendite* per la richiesta (i, q) da customer quando nella risposta (i, k) dal server si ha $k < q$, cioè non tutti gli items richiesti vengono forniti.

1 Obiettivo

Il numero di mancate vendite al tempo t è il numero totale di mancate vendite entro il tempo t . Il numero di transazioni al tempo t è il numero totale di transazioni effettuate entro il tempo t .

Il rate di mancate vendite al tempo t è il numero di mancate vendite al tempo t diviso il numero di transazioni al tempo t .

Il rate di mancate vendite è il rate di mancate vendite alla fine della simulazione.

Si vuole riportare in uscita il valore atteso del rate di mancate vendite. Cioè il valore medio del rate di mancate vendite usando M simulazioni Montecarlo.

2 Parametri

I parametri della simulazione sono:

1. H: orizzonte di simulazione (valore reale positivo)
2. M: numero di simulazioni montecarlo (valore intero positivo)

3. C: numero di customers (valore intero positivo)
4. S: numero di servers (valore intero positivo)
5. F: numero di fornitori (valore intero positivo)
6. A: parametro A del testo (valore reale positivo)
7. B: parametro B del testo (valore reale positivo)
8. V: parametro V del testo (valore reale positivo)
9. W: parametro W del testo (valore reale positivo)
10. P: numeri prodotti (valore intero positivo)
11. Q: parametro Q nel testo (valore intero positivo)
12. r: tempo lettura rete (valore reale positivo)
13. w: tempo scrittura rete (valore reale positivo)
14. l: tempo lettura DB (valore reale positivo)
15. s: tempo scrittura DB (valore reale positivo)
16. z: tempo lettura server (valore reale positivo)
17. v: tempo scrittura server (valore reale positivo)

3 Formato dei parametri di input

I parametri della simulazione sono forniti nel file `parameters.txt`. Le righe del file `parameters.txt` hanno tutte lo stesso formato:

<ID del parametro> <valore del parametro>

Un esempio di file `parameters.txt` è:

```
H 234.6
M 100
C 10
A 1.0
B 2.0
Q 3
V 3.0
W 5.0
P 10
S 4
F 2
l 3.1
```

```
s 6.3
r 0.01
w 0.01
z 0.1
v 0.2
```

4 Formato di output

L'output dell'esercizio è memorizzato nel file `results.txt` la cui prima riga è formattata come indicato nelle istruzioni generali.

Le rimanenti righe del file `results.txt` hanno il formato

R < rate mancate vendite >

Un esempio di file `results.txt` per il file `parameters.txt` dato sopra è:

```
2025-01-09-Mario-Rossi-1234567
R 12.3
```

5 Struttura Sorgenti

Deve esserci almeno una classe, in un proprio file `cpp`, per ognuno dei processi: `customer`, `server`, `supplier`, `network`, `db`. Quindi devono esserci almeno i seguenti files: `customer.cpp`, `server.cpp`, `supplier.cpp`, `network.cpp`, `db.cpp`.